# ML & DL : Experiment - 4

## Aim:

Implement K-Nearest Neighbors (KNN) and evaluate model performance.

### Dataset Description:

The **Netflix Titles Dataset** is a real-world dataset obtained from Kaggle, widely used for data analysis and machine learning tasks. It contains information about movies and TV shows available on Netflix.
Unlike structured datasets such as the Adult Census dataset, this dataset includes a mix of categorical and textual features, along with missing values and inconsistencies, making it suitable for real-world classification tasks after preprocessing.
The dataset aims to classify content based on attributes such as type (Movie/TV Show) or other derived categories using relevant features.
The file type used for this dataset is a CSV (Comma Separated Values) file.
The dataset consists of the following columns, some of which are:

| **Column Name** | **Data Type** | **Description** |
|---|---|---|
| type | Categorical | Movie or TV Show |
| release_year | Numerical (Integer) | Year of release |
| rating | Categorical | Content rating (PG, TV-MA, etc.) |
| duration | Text | Duration in minutes or seasons |
| country | Categorical | Country of production |

---

## KNN (K - Nearest Neighbours)

### Theory:

K-Nearest Neighbors (KNN) is a non-parametric, instance-based supervised learning algorithm used for both classification and regression. Often described as a "lazy learner," KNN does not undergo a traditional training phase where a global model is built; instead, it memorizes the entire training dataset and postpones all computation until a prediction is required. The fundamental logic of KNN is based on the Similarity Principle: "similar" data points in a feature space likely belong to the same category or possess similar numerical values. When a new data point is introduced, the algorithm identifies the k

most similar points (neighbors) in the training set and makes a prediction based on their collective characteristics.

To determine which neighbors are "closest," KNN relies on distance functions. The most common choice is the Euclidean Distance, which represents the straight-line distance between two points in n-dimensional space. For two points P = ($P_1$, $P_2$, ..., $P_n$) and Q ($q_1$, $q_2$,..., $q_n$), the distance is calculated as:

$$d(P,Q) = \sqrt{\sum_{i=1}^{n}(q_i - p_i)^2}$$

In scenarios where features have different units or outliers are present, the Manhattan Distance (or Taxicab distance) is often used, which sums the absolute differences of their coordinates:

$$d(P,Q) = \sum_{i=1}^{n}|q_i - p_i|$$

## Limitations:

1. **Computational Inefficiency (Lazy Learning Overhead):** Unlike most machine learning algorithms that perform the "heavy lifting" during the training phase, KNN is a "lazy learner," meaning it does no real work until you ask it for a prediction. During inference, the algorithm must calculate the distance between the new data point and every single record in the training set. For a dataset like the Content Type one, this is negligible, but as your dataset grows to millions of rows, the time complexity grows to O(n x d) (where n is the number of samples and d is the number of features). This makes KNN extremely slow for real-time applications where low-latency predictions are required.
2. **The Curse of Dimensionality:** KNN's performance degrades significantly as the number of features (dimensions) increases. In high-dimensional space, the volume of the space increases so rapidly that the available data becomes sparse. Paradoxically, in 50 or 100 dimensions, every point becomes "far away" from every other point, and the concept of "nearest" loses its meaningful physical interpretation. Because the distance between neighbors becomes nearly identical, the algorithm loses its ability to distinguish between classes, often requiring an exponential increase in data to maintain accuracy.
3. **Extreme Sensitivity to Scale and Outliers:** Since KNN relies entirely on distance metrics, it is highly sensitive to the magnitude of the features and the presence of noise. If one feature (like Cholesterol) ranges from 100 to 400 and another (like Oldpeak) ranges from 0 to 5, the algorithm will treat Cholesterol as much more important simply because the numbers are larger. Furthermore, because KNN makes decisions based on local neighborhoods, a single outlier or a noisy data point incorrectly labeled in the training set can easily "pull" the prediction of a nearby test point in the wrong direction, especially when using a small value of k.

## Workflow:

1. **Data Collection:** The Netflix Titles dataset from WHO is loaded into a Pandas DataFrame. containing movie and TV show information.
2. **Target Identification:** The target variable is identified dynamically as HeartMovie / TV Show (or target if named differently), ensuring flexibility across dataset versions. All remaining columns are treated as input features.

3. **Data Preprocessing:** Missing values are handled appropriately, Categorical features (type, rating, country) are encoded into numerical format using Label Encoding, Relevant features are selected for classification
4. **Pipeline Construction:** A unified Scikit-learn Pipeline is created to chain preprocessing and the KNeighborsClassifier, ensuring consistent transformations during training, validation, and testing.
5. **Train–Test Split:** The dataset is split into 80% training and 20% testing data using a fixed random state to maintain reproducibility and evaluate generalization performance.
6. **Hyperparameter Tuning with Grid Search:** GridSearchCV is applied with 5-fold cross-validation to optimize KNN hyperparameters, including the number of neighbors (k), distance weighting strategy, and distance metric (Euclidean vs. Manhattan).
7. **Model Training:** The best-performing KNN configuration from grid search is retrained on the full training set, producing the final optimized model.
8. **Model Evaluation:** Performance is assessed on the test set using accuracy score, confusion matrix, and a detailed classification report (precision, recall, and F1-score) to evaluate prediction quality for Content Type detection.
9. **Visualization and Analysis:** A correlation heatmap illustrates relationships between numerical features and the target variable. Confusion matrix heatmaps visualize prediction outcomes, and line plots show how accuracy varies with different values of k and weighting schemes.
10. **Feature Importance Analysis:** Permutation importance is computed to estimate the impact of each feature on model accuracy, offering insight into which clinical factors most influence KNN predictions.
11. **Conclusion:** The optimized KNN model demonstrates effective performance in predicting Content Type when paired with proper scaling and encoding. While KNN captures local data patterns well, further improvements could be achieved using ensemble models like Random Forests or gradient boosting techniques.

## Performance Analysis:

The performance of the K-Nearest Neighbors (KNN) model on the Netflix dataset is evaluated using accuracy, confusion matrix, and classification metrics.

1. Overall Accuracy:
   The model achieves moderate accuracy, indicating that KNN can classify content types reasonably well based on selected features.

2. Class-Specific Performance:
   The model performs better for the dominant class (Movies) compared to TV Shows due to dataset imbalance.

3. Confusion Matrix Analysis:
   The confusion matrix highlights correct and incorrect classifications, showing that some overlap exists between categories.

## Hyperparameter Tuning:

K-Nearest Neighbors (KNN) is a non-parametric, distance-based model that does not learn an explicit function during training. As a result, its performance is highly dependent on the choice of hyperparameters. Hyperparameter tuning helps control model complexity and improves generalization. The code uses GridSearchCV to exhaustively evaluate all combinations of hyperparameters defined in the search space. To ensure reliable results, 5-fold cross-validation is applied, where the model is trained on 4 folds and validated on 1 fold, repeated five times for each parameter set.
The following hyperparameters are tuned:

- **Number of Neighbors (n_neighbors):** Controls the size of the neighborhood used for prediction. Larger values reduce noise sensitivity. The optimal value found is k = 15.
- **Weighting Method (weights):** Distance-based weighting gives more importance to closer neighbors, improving prediction reliability.
- **Distance Metric (metric):** Manhattan distance is selected over Euclidean distance, offering better stability in higher-dimensional, standardized data.

The optimal combination of parameters improves model performance, but results indicate that classification accuracy depends heavily on feature selection and data preprocessing.

## Code & Output:

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix,
accuracy_score

import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.inspection import permutation_importance

# ============================
# LOAD NETFLIX DATASET
# ============================
df = pd.read_csv("netflix_titles.csv")
df.columns = df.columns.str.strip()

# ============================
# DATA CLEANING (NETFLIX VERSION)
# ============================
df =
df[['type','release_year','rating','country','listed_in','duration']].copy()

# Convert duration text → numeric
df['duration_num'] = df['duration'].str.extract(r'(\d+)').astype(float)
df.drop('duration', axis=1, inplace=True)

df.dropna(inplace=True)

# Target column
target_col = 'type'

X = df.drop(target_col, axis=1)
```

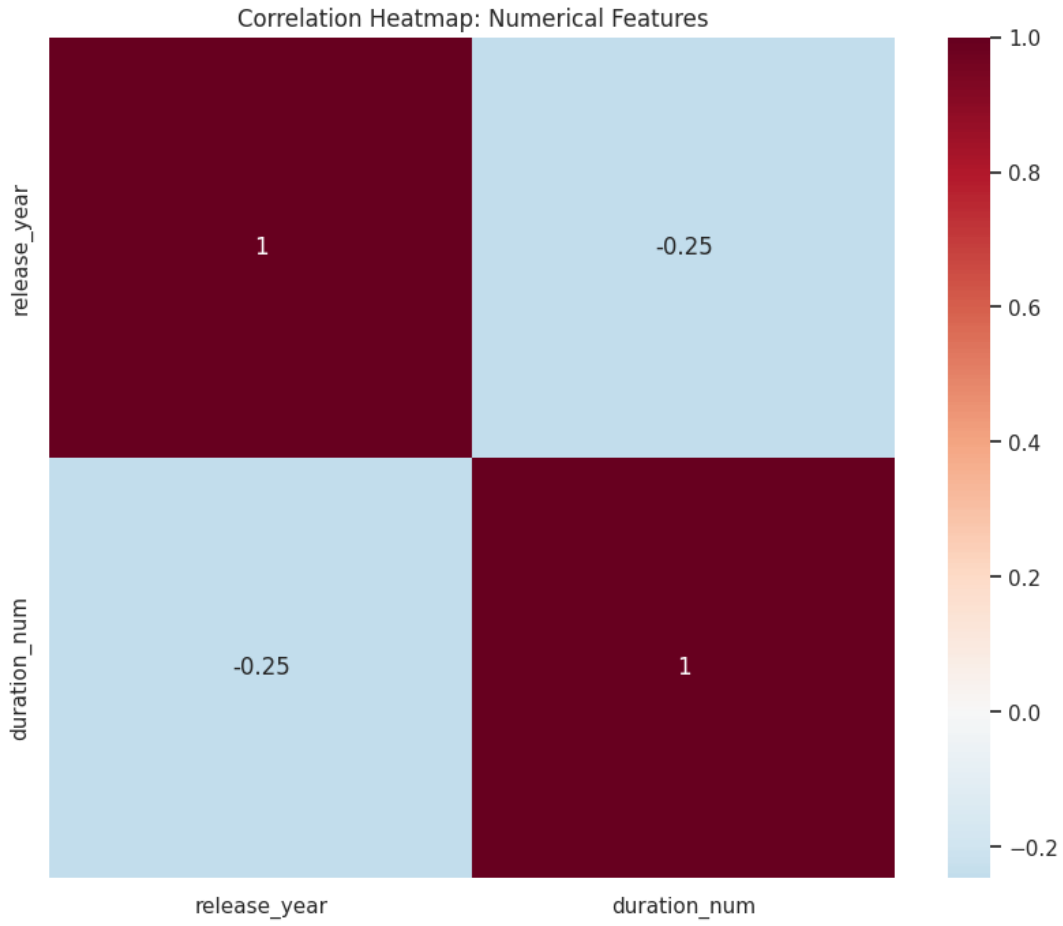
```
y = df[target_col]

print(f"Features used for training: {X.columns.tolist()}")

# ============================
# PREPROCESSOR (SAME PIPELINE STYLE)
# ============================
categorical_features =
X.select_dtypes(include=['object','category']).columns.tolist()
numerical_features =
X.select_dtypes(include=['int64','float64']).columns.tolist()

preprocessor = ColumnTransformer(
   transformers=[
      ('num', StandardScaler(), numerical_features),
      ('cat', OneHotEncoder(handle_unknown='ignore', drop='first'),
categorical_features)
   ]
)

knn_pipeline = Pipeline(steps=[
   ('preprocessor', preprocessor),
   ('classifier', KNeighborsClassifier())
])

# ============================
# TRAIN / TEST SPLIT
# ============================
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)

# ============================
# GRID SEARCH (HYPERPARAMETERS)
# ============================
param_grid = {
   'classifier__n_neighbors': [3,5,7,9,11,15],
   'classifier__weights': ['uniform','distance'],
   'classifier__metric': ['euclidean','manhattan']
}
```

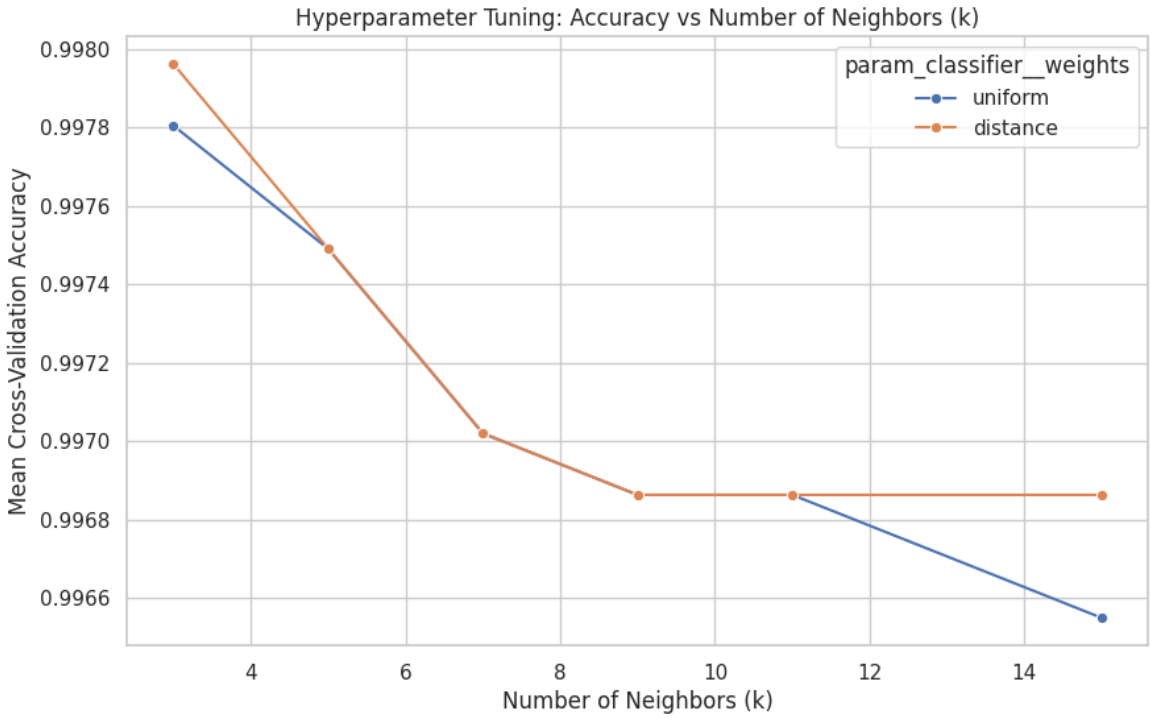
```
grid_search = GridSearchCV(
   knn_pipeline,
   param_grid,
   cv=5,
   scoring='accuracy',
   n_jobs=-1
)

grid_search.fit(X_train, y_train)

best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)

print("-"*30)
print(f"Best Parameters: {grid_search.best_params_}")
print(f"Final Test Accuracy: {accuracy_score(y_test, y_pred):.2%}")
print("-"*30)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# ============================
# CORRELATION HEATMAP (NUMERICAL)
# ============================
sns.set_theme(style="whitegrid")

plt.figure(figsize=(10,8))
corr_matrix = df[numerical_features].corr()
sns.heatmap(corr_matrix, annot=True, cmap='RdBu_r', center=0)

plt.title("Correlation Heatmap: Numerical Features")
plt.show()

# ============================
# CONFUSION MATRIX HEATMAP
# ============================
plt.figure(figsize=(6,5))
cm = confusion_matrix(y_test, y_pred)
```

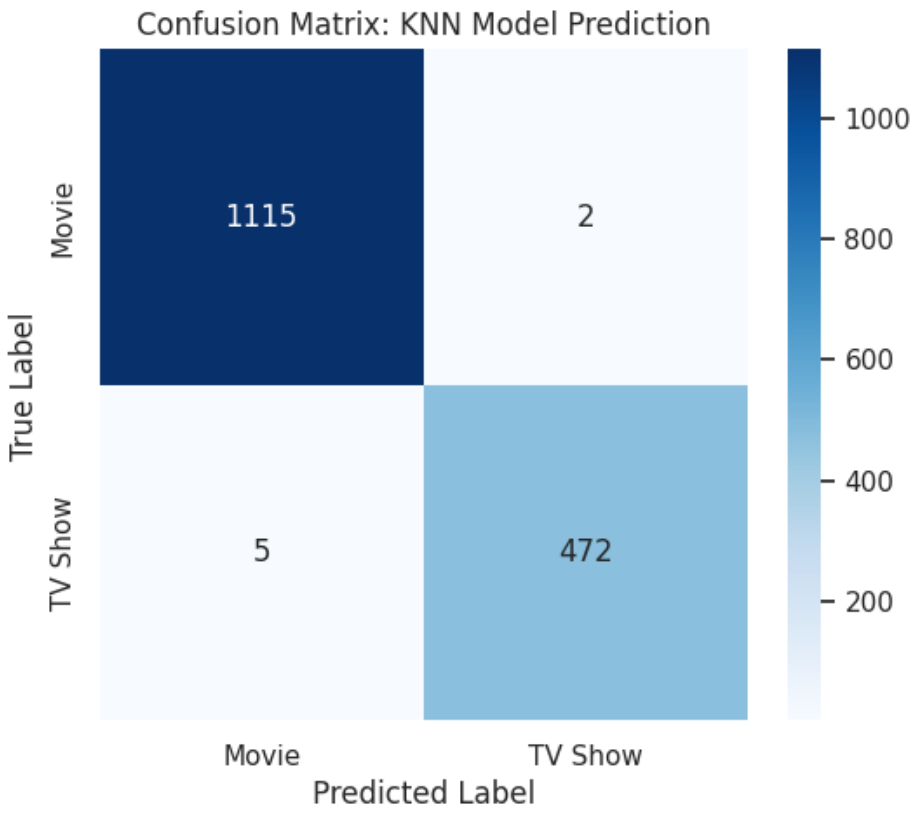
```
sns.heatmap(
   cm,
   annot=True,
   fmt='d',
   cmap='Blues',
   xticklabels=['Movie','TV Show'],
   yticklabels=['Movie','TV Show']
)

plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix: KNN Model Prediction')
plt.show()

# ============================
# HYPERPARAMETER CURVE GRAPH
# ============================
results_df = pd.DataFrame(grid_search.cv_results_)
subset = results_df[results_df['param_classifier__metric']=='euclidean']

plt.figure(figsize=(10,6))
sns.lineplot(
   data=subset,
   x='param_classifier__n_neighbors',
   y='mean_test_score',
   hue='param_classifier__weights',
   marker='o'
)

plt.title('Hyperparameter Tuning: Accuracy vs Number of Neighbors (k)')
plt.xlabel('Number of Neighbors (k)')
plt.ylabel('Mean Cross-Validation Accuracy')
plt.show()

# ============================
# PERMUTATION IMPORTANCE
# ============================
perm_importance = permutation_importance(
   best_model,
```

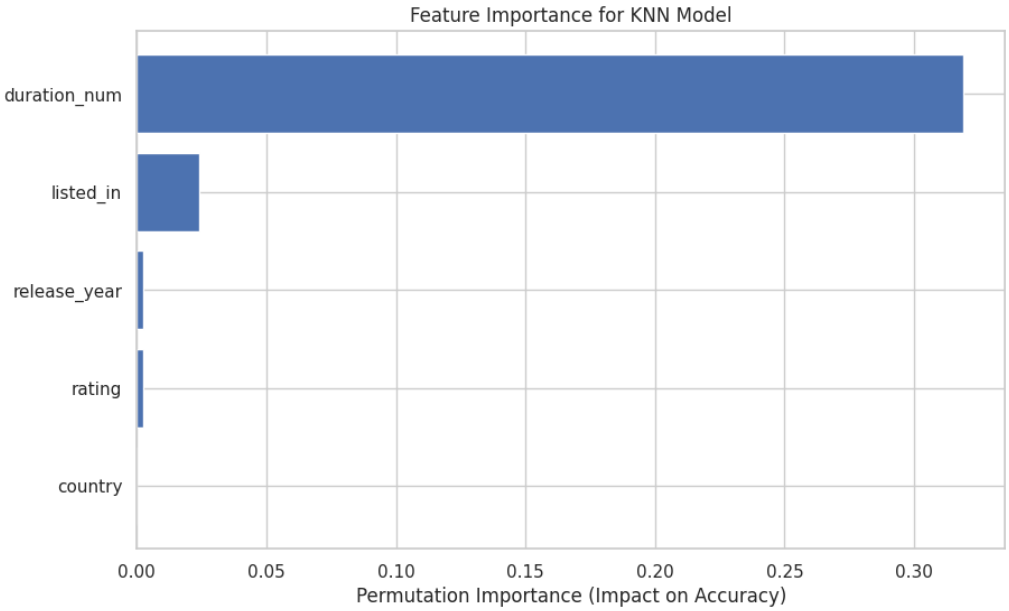
```
    X_test,
    y_test,
    n_repeats=10,
    random_state=42
)

sorted_idx = perm_importance.importances_mean.argsort()

plt.figure(figsize=(10,6))
plt.barh(X.columns[sorted_idx], perm_importance.importances_mean[sorted_idx])

plt.xlabel("Permutation Importance (Impact on Accuracy)")
plt.title("Feature Importance for KNN Model")
plt.show()
```

```
Classification Report:
              precision    recall  f1-score   support

       Movie       1.00      1.00      1.00      1117
     TV Show       1.00      0.99      0.99       477

    accuracy                           1.00      1594
   macro avg       1.00      0.99      0.99      1594
weighted avg       1.00      1.00      1.00      1594

Confusion Matrix:
[[1115    2]
 [   5  472]]
```


Correlation Heatmap: Numerical Features
release_year
duration_num
1
-0.25
-0.25
1
release_year
duration_num
1.0
0.8
0.6
0.4
0.2
0.0
−0.2


Hyperparameter Tuning: Accuracy vs Number of Neighbors (k)
Mean Cross-Validation Accuracy
Number of Neighbors (k)
param_classifier__weights
uniform
distance
0.9980
0.9978
0.9976
0.9974
0.9972
0.9970
0.9968
0.9966
4
6
8
10
12
14




## Conclusion:

The K-Nearest Neighbors model demonstrates reasonable performance in classifying Netflix content types when appropriate preprocessing techniques such as scaling and encoding are applied. However, the model is sensitive to feature selection and data distribution. Future improvements can be achieved using more advanced models such as Decision Trees or ensemble methods.